

Desafio: API REST para Livraria

Construir uma API REST em **.NET 8** para gerenciar livros de uma livraria, com CRUD completo, validações, Swagger, regras de negócio, cache, logs, Health Checks e boas práticas de arquitetura.

Requisitos:

1. Criar livro; Listar livros; Buscar livro por ID; Atualizar livro; Excluir livro.
2. Filtro por título; Filtro por autor; Filtro por gênero; Filtro por faixa de preço; Paginação; Ordenação por título, preço ou data de criação.
3. .NET 8;
4. Entity Framework Core;
5. SQL Server;
6. Migrations;
7. Postman;
8. FluentValidation;
9. Serilog;
10. Health Checks;
11. Redis Cache;
12. Repository Pattern;
13. Unit of Work;
14. Testes unitários com xUnit e Moq;
15. Docker;
16. Autenticação com JWT;
17. Autorização por perfil: Admin e User;

Endpoints obrigatórios:

Método	Endpoint	Descrição
POST	/api/books	Criar livro
GET	/api/books	Listar livros
GET	/api/books/{id}	Buscar por ID
PUT	/api/books/{id}	Atualizar livro
DELETE	/api/books/{id}	Excluir livro

Método	Endpoint	Descrição
GET	/api/books/search	Buscar por título, autor ou gênero
PATCH	/api/books/{id}/stock	Atualizar estoque
PATCH	/api/books/{id}/price	Atualizar preço

Campos obrigatórios:

Campo	Tipo	Regra
Id	Guid	Gerado automaticamente
Title	string	2 a 120 caracteres
Author	string	2 a 120 caracteres
Genre	string/enum	Deve ser válido
Price	decimal	Maior ou igual a 0
Stock	int	Maior ou igual a 0
CreatedAt	DateTime	Preenchido na criação
UpdatedAt	DateTime?	Atualizado nas alterações

Campo	Tipo	Regra
ISBN	string	Obrigatório e único
Publisher	string	Obrigatório
PublishedYear	int	Não pode ser maior que o ano atual
IsActive	bool	Usado para soft delete
Description	string	Opcional, máximo 1000 caracteres

Regras de negócio:

- Não permitir livro duplicado com mesmo Title + Author;

- Não permitir ISBN duplicado;
- Price não pode ser negativo;
- Stock não pode ser negativo;
- Genre deve ser um enum válido;
- PublishedYear não pode ser futuro;
- Exclusão deve ser soft delete, alterando IsActive = false;
- Listagens devem retornar apenas livros ativos;
- Não permitir atualizar livro inativo;
- Quando atualizar, preencher UpdatedAt;
- Ao criar livro, limpar cache da listagem;
- Ao atualizar ou excluir livro, limpar cache relacionado.
- Apenas Admin pode criar, atualizar e excluir livros;
- Rate limiting;
- Auditoria de alterações;
- Rate limiting; Correlation ID;

Health Checks

Criar endpoint:

GET /health

Deve verificar:

- API online;
- Conexão com SQL Server;
- Conexão com Redis.

Exemplo esperado:

```
{
  "status": "Healthy",
  "checks": [
    {
      "name": "sqlserver",
      "status": "Healthy"
    },
  ],
}
```

```
{  
  "name": "redis",  
  "status": "Healthy"  
}  
]  
}
```

Redis Cache

Usar Redis para cache em:

- **GET /api/books;**
- **GET /api/books/{id};**
- **GET /api/books/search.**

Regras:

- **Cache da listagem por 5 minutos;**
- **Cache do livro por ID por 10 minutos;**
- **Ao criar, atualizar ou excluir livro, invalidar cache;**
- **Usar chave padronizada, exemplo:**

books:all:page:1:size:10

books:id:{id}

books:search:{term}